OptimizedTrafficCLIP: Reproducibility, Validation, and Extension of a Tri-Modal Framework for Encrypted Network Traffic Classification

Sahil Dhanuka, Vishrav Asthana, Shambhav Ghosh, Dhyanam Mahajan, and Naresh Kumar Bishnoi

***Abstract*—We present an independent reproducibility study of OptimizedTrafficCLIP, a tri-modal framework for encrypted traffic classification that fuses a byte-image branch, a flow-statistics branch, and physics-conditioned text prompts. A line-by-line audit of the released code identifies six defects, including an evaluation-time leak of the ground-truth label into the model's text input. After all six are corrected, the released implementation reaches 88.29% ± 2.15% Macro F1 over five seeds under the manuscript's own protocol, above the manuscript's single-run 79.63%, while its separately reported 87.54% headline figure does not reproduce. Two clean-room reimplementations, one bi-modal and one tri-modal, reach 99.7% accuracy on the same 10-class USTC-TFC2016 task, 11.6 points above the corrected release, and 96.6% at 20 classes, indicating that the architecture is not the limiting factor. Four from-scratch baselines exceed 99% accuracy on the 10-class benchmark, showing that the task is largely solved by several unrelated model families; re-training two of them on their own published input representations shows that the input representation alone moves accuracy by up to 29 points. The residual error of the correct tri-modal model concentrates in one pair of botnet families whose flow statistics are indistinguishable. The bi-modal and tri-modal parity result replicates on three further datasets covering VPN, 5G, and IoT traffic. We release a public reproducibility package and distill the study into a five-stage audit procedure and eleven recommendations for reproducible research in encrypted traffic classification.**

***Index Terms*—Encrypted traffic classification, reproducibility, network measurement, multimodal fusion, physics-informed prompting, network security.**

I. Introduction

Network traffic classification underpins a broad range of security and quality-of-service objectives in modern network management, including intrusion detection, application-aware routing, policy enforcement, and capacity planning. The problem has become substantially harder as encryption has become the default: payload inspection is defeated by encryption, and application-layer signatures are increasingly obfuscated by tunneling and overlay protocols.

Deep learning approaches have become the dominant response to these challenges. One line of work converts raw packet sequences into fixed-size byte images and applies convolutional networks to the resulting visual patterns. A parallel line treats each flow as a sequence and applies recurrent or transformer encoders to packet-level or byte-level tokens. A more recent line borrows the contrastive language-image pretraining (CLIP) paradigm from computer vision, projecting both a visual encoding of the flow and a textual encoding of the candidate class label into a shared embedding space and classifying by similarity.

OptimizedTrafficCLIP [9] extends this cross-modal line of work with a third modality: session-level physics statistics, namely mean inter-arrival time, jitter, and byte entropy, used both as a standalone embedding and to condition a dynamic, per-sample text prompt. A non-linear fusion head then replaces the cosine-similarity classifier of the base CLIP-style design. The released manuscript reports a Macro F1 score of 87.54% on a 10-class subset of USTC-TFC2016, a substantial improvement over its reported visual-only configurations.

A reported number is only as trustworthy as the pipeline that produced it, and a reproducibility study that stops at flagging a discrepancy leaves open whether the discrepancy is a coding error or a property of the design being tested. This paper goes one step beyond a pass/fail replication verdict: it audits the released OptimizedTrafficCLIP implementation, corrects every defect found in it, re-measures the corrected code under a statistically honest multi-seed protocol rather than the manuscript's own single-run report, isolates whether any remaining gap against two independent from-scratch reimplementations is architectural or implementation-specific, and traces the error that persists even in a defect-free reimplementation to its root cause rather than leaving it unexplained. The contribution of this work is methodological rather than architectural: it proposes no new model, and instead establishes how much of a published result survives independent verification and why. Specifically, the contributions are as follows.

- A line-by-line audit of the released training and evaluation pipeline, identifying six defects, one of which is a direct ground-truth label leak in the evaluation path.
- An honest re-evaluation of the released code under the manuscript's own evaluation protocol, after all six defects are fixed in place, averaged over five independent seeds with a pairwise significance test reported alongside its own power limitation rather than collapsed into a single point estimate.
- An independent bi-modal reimplementation of the underlying TrafficCLIP design, built from the published specification without reference to the released code, to establish a defect-free reference point.

The authors are with the School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore 632014, India (registration numbers 24BDS0456, 24BAI0354, 24BCB0013, 24BCE2531, 24BDS0336).

This work was carried out under the guidance of Pramit De, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India.

- A second independent reimplementation that reproduces OptimizedTrafficCLIP's full tri-modal design, free of the identified defects by construction, to isolate whether the corrected release's residual gap is architectural or implementation-specific.
- Four independent baseline models, FlowPic, FS-Net, a 1D-CNN, and a small Transformer, trained from scratch on the identical data and test partition, to contextualize all of the above against architecturally unrelated prior approaches, with FlowPic and FS-Net additionally retrained on each model's own originally published input representation to close the byte-image substitution's scope caveat quantitatively.
- A root-cause analysis of the residual 20-class error that remains in the defect-free tri-modal reimplementation, isolating it to a single confusable class pair and showing that the available feature set cannot separate them, rather than leaving the remaining gap unexplained.
- A cross-dataset generalization test of both independent reimplementations and all four baselines on three additional, independently sourced datasets spanning VPN traffic-type, 5G network-slice, and IoT device/malware-family classification, showing that the bi-modal/tri-modal parity result is not specific to USTC-TFC2016.
- A reusable five-stage audit procedure for released traffic classifiers (Section IV-A), together with concrete recommendations for reproducible research in encrypted traffic classification derived from the defects found here (Section IX), and a public reproducibility package containing every corrected pipeline, independent reimplementation, baseline, and per-seed result reported in this paper.

II. Related Work

This study sits at the intersection of four lines of work: encrypted traffic classification models, cross-modal and multimodal traffic models, reproducibility and evaluation practice in machine learning, and evaluation pitfalls specific to machine learning for networking and security. We review each in turn and close with the gap this paper addresses.

A. Encrypted Traffic Classification

Early traffic classification relied on transport-layer port numbers, a method defeated once applications began using dynamic or randomized ports. Deep packet inspection followed, matching payload byte patterns against known application signatures, an approach rendered ineffective once encryption became the norm for user traffic. A subsequent generation of work extracted handcrafted statistical features such as packet size distributions, timing statistics, and flow duration, and applied classical machine learning models such as random forests and support vector machines. These models avoid reading payload content but require domain expertise to design the right features, and the resulting features transfer poorly across networks.

Deep learning removed the manual feature engineering step along two separate paths. The first path converts a flow's raw bytes into a fixed-size image and applies a convolutional network. FlowPic is a representative example, framing traffic classification as an image recognition problem over a two-dimensional histogram of packet sizes and inter-arrival times [5]. The dataset used throughout this paper, USTC-TFC2016, originates from a similar image-based convolutional classification approach [3]. The second path treats a flow as a sequence: FS-Net encodes bidirectional packet-length sequences with a gated recurrent encoder-decoder, trained jointly with a reconstruction objective and a classification head [6]. Both families reach strong accuracy on their respective benchmarks but do not combine visual and sequential evidence within a single model.

Pretraining now dominates the recent literature. YaTC [18] pretrains a traffic Transformer with a masked-autoencoder objective over a multi-level flow representation. NetMamba [19] replaces the Transformer with a unidirectional state-space model to cut inference cost. TrafficFormer [20] adds a fine-grained multi-classification pretraining task and a field-level augmentation scheme for fine-tuning. Each reports accuracy close to or above 99% on public benchmarks that include USTC-TFC2016 or ISCXVPN2016. Our Track C results are consistent with those high ceilings: four comparatively small models trained from scratch, with no pretraining, also exceed 99% on the 10-class USTC-TFC2016 task. That ceiling is the relevant context for any single model's reported gain on this benchmark.

B. Cross-Modal and Multimodal Traffic Models

A third line of work adapts the contrastive language-image pretraining paradigm [8] to non-visual domains by pairing a data encoder with a frozen or lightly adapted text encoder such as BERT [2], and classifying by embedding similarity rather than a closed-set softmax head. TrafficCLIP [1] applies this idea to network traffic: a multi-level visual encoder, combining a trainable detail-aware branch with a frozen pretrained backbone [4], is paired with a frozen BERT text branch encoding each class name as a static prompt, and the two embeddings are compared by cosine similarity after a lightweight adapter. A related pretraining line skips the CLIP-style dual encoder and instead pretrains a single transformer directly on datagram bytes: ET-BERT [10] pretrains on unlabeled traffic and fine-tunes for classification, reaching strong results without any hand-designed statistical branch, which frames physics-informed prompting as one alternative among several for injecting traffic-specific structure into a transformer-based classifier rather than the only viable one. OptimizedTrafficCLIP builds on the CLIP-style design by adding a third, physics-informed statistical modality, replacing the static class-name prompt with a per-sample dynamic prompt conditioned on the flow's own statistics, and replacing the cosine-similarity classifier with a trainable non-linear fusion head. This paper treats OptimizedTrafficCLIP as the system under study and TrafficCLIP as the architectural baseline against which its tri-modal extension is isolated.

Contrastive alignment between traffic and text continues to be explored: FlowCLIP [21] aligns flow features with domain

names rather than class-name prompts. Multimodal fusion for traffic predates the CLIP line. MIMETIC [22] fuses a payload-byte branch and a packet-header-field branch for mobile encrypted traffic and learns intra- and inter-modality dependencies jointly. A generic multi-modal autoencoder pipeline has been proposed to unify classification, anomaly detection, and novelty discovery under one architecture [14], and Traffic-CBM [23] organizes multimodal traffic evidence into interpretable concept spaces. None of these works includes an ablation under which the added modality can be isolated against a matched single- or bi-modal counterpart built independently. Our Track A and Track A-Full pairing provides exactly that test for one tri-modal design and finds parity on four datasets.

C. Reproducibility in Machine Learning

Concern about reproducibility in machine learning is now well documented. The NeurIPS 2019 reproducibility program introduced a code-submission policy, a community reproducibility challenge, and a reproducibility checklist [24]. A survey of machine-learning-based science found data leakage in at least 294 papers across 17 fields and proposed a taxonomy of eight leakage types together with model info sheets to detect them [25]; the ground-truth label leak identified in this paper (Defect 2 of Table II) is an instance of one of those types, leakage from the test label into the model input at evaluation time. Benchmark variance is a second, independent source of irreproducibility: variance from data sampling, parameter initialization, and hyperparameter choice can exceed the gaps that papers report between methods, so single-run comparisons are unreliable [26]. That finding motivates our multi-seed protocol for every track, including the released code, which was originally reported from a single run.

D. Evaluation Pitfalls in ML for Networking and Security

Security and networking applications of machine learning have their own documented pitfalls. Arp *et al.* identify ten recurring pitfalls, including sampling bias, label inaccuracy, data snooping, and inappropriate baselines, and find them in a large fraction of the security papers they review [27]. Jacobs *et al.* show that network-security classifiers with near-perfect reported accuracy frequently rely on shortcuts that an explanation of the trained model exposes [28]. Specific to encrypted traffic, a systematization of the literature finds that many studies rely on outdated or improperly filtered datasets [11]; Zhao *et al.* show that the strong results of pretrained representation-learning models are driven largely by data-preparation flaws that create spurious correlations between features and labels, and collapse when those shortcuts are removed [29]; BiasSeeker [30] detects such environment-entangled shortcut features automatically; and an audit of fourteen encrypted-traffic benchmarks finds systematic problems in how flow labels are derived [31]. On the infrastructure side, DataZoo [12] standardizes dataset access and partitioning so that results across papers become comparable, and re-running published models across each other's datasets exposes context-specific assumptions that a single-split evaluation does not surface [13]. The networking community also publishes replications directly: an IMC replication of contrastive learning on the FlowPic representation reproduced the original findings but uncovered a 20% accuracy drop on some scenarios caused by a data shift in the original dataset [32].

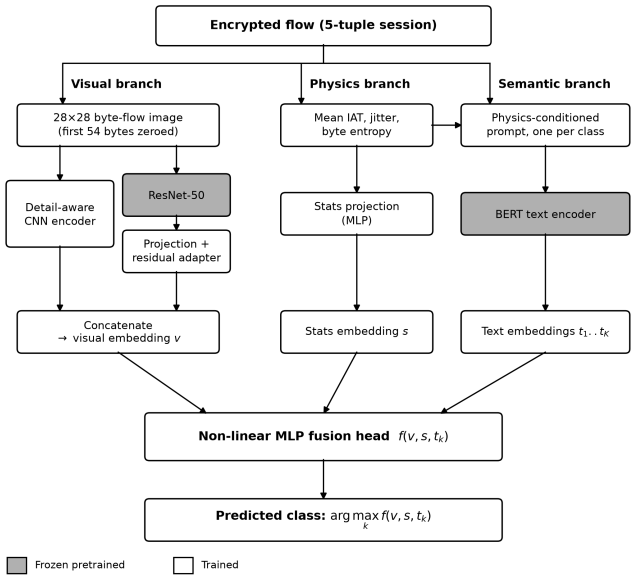


Fig. 1. Tri-modal OptimizedTrafficCLIP architecture. Grey blocks are frozen pretrained modules; white blocks are trained. The detail-aware encoder and the ResNet-50 path run in parallel and are concatenated.

E. Gap Addressed by This Work

Prior work in this space falls into two groups. Corpus-level studies [11], [27], [29], [31] diagnose recurring problems across many papers but do not correct any one implementation and re-measure it. Replications [32] re-run a method but do not audit its released code line by line. To our knowledge, no prior study of an encrypted-traffic classifier combines four things: a code-level audit of the released implementation, an in-place correction and multi-seed re-evaluation under the authors' own protocol, independent clean-room reimplementations that isolate architecture from implementation, and external baselines and additional datasets that place the corrected result in context. This paper does all four for OptimizedTrafficCLIP and distills them into a reusable audit procedure (Section IV-A) and a set of recommendations (Section IX). Its contribution is methodological. It proposes no new model.

III. The OptimizedTrafficCLIP Architecture

Fig. 1 summarizes the tri-modal architecture under study. A raw encrypted flow, keyed by its 5-tuple, is consumed by three parallel branches whose outputs are combined by a shared fusion head.

A. Visual Branch

The first 784 bytes of each flow are reshaped into a 28×28 grayscale image, with the first 54 header bytes zeroed to remove trivially identifying fields. A trainable detail-aware convolutional encoder is combined with a semantics-aware

TABLE I
FIVE-STAGE AUDIT PROCEDURE

Stage	Question	Instantiation here
S1	Does the released code implement the stated protocol?	Line-by-line audit; six defects (Table II)
S2	What does the released code score once corrected?	In-place fixes, manuscript's own protocol, 5 seeds (Track B)
S3	Is a remaining gap architectural or implementation-specific?	Clean-room bi-modal and tri-modal rebuilds (Tracks A, A-Full)
S4	How hard is the task, and does the finding transfer?	Four external baselines; three additional datasets (Track C)
S5	Where does the residual error come from?	Per-class and confusable-pair analysis, feature probe

branch built on a pretrained ResNet-50 backbone [4], adapted from three-channel to single-channel input and kept frozen. A lightweight traffic adapter and a residual connection combine the two sub-branches into a single visual embedding.

B. Physics Branch

Three session-level statistics, mean inter-arrival time, jitter, and byte entropy, are computed per flow and projected through a trainable stats projection head into a stats embedding. These same three statistics also condition the dynamic prompt consumed by the semantic branch.

C. Semantic Branch

Rather than a single static prompt per class name, each candidate class is rendered as a short natural-language description conditioned on the sample's own physics statistics. The resulting text is encoded by a frozen pretrained BERT model [2], and the K candidate embeddings, one per class, are scored against the fused vision-and-stats representation at inference time.

D. Fusion and Training Objective

A non-linear multilayer perceptron fusion head combines the visual, stats, and text embeddings and produces a class score, replacing the cosine-similarity classifier of the base TrafficCLIP design. The network is trained with a joint objective combining cross-entropy loss and a supervised contrastive loss term [7], weighted by a coefficient α.

IV. METHODOLOGY

A. Audit Procedure Overview

The study follows a five-stage procedure. Each stage answers one question that the previous stages leave open, and each can be reused for auditing other released traffic classifiers. Table I lists the stages and where each is carried out in this paper.

S1 separates defects in the code from properties of the design. S2 measures what the released artifact actually achieves after correction, using the authors' own protocol so that the comparison with their reported table is like for like. S3 is the step most reproducibility studies omit: if the corrected release still falls short of a reference, only an implementation built without reference to the released code can tell whether the architecture or the implementation is responsible. S4 fixes the scale against which every other number is read, since a gain on a benchmark that unrelated models already saturate carries little information. S5 attributes the error that survives all corrections, so that the remaining gap is explained rather than left as an open discrepancy. Every stage shares one data split, verified identical across implementations by an automated test, and reports multi-seed statistics.

B. Dataset and Split

All experiments use USTC-TFC2016 [3], preprocessed identically across every track compared in this paper: flows are keyed by 5-tuple, the first 784 bytes are reshaped into a 28×28 grayscale image, and the first 54 header bytes are zeroed. Both a 10-class balanced subset at 900 samples per class and a 20-class extension are evaluated. All tracks use the same stratified 70/15/15 train, validation, and test partition at a fixed seed. An automated test asserts directly that the split indices generated independently by each implementation are identical for the same input archive, so the comparisons reported below are not an artifact of differing test sets.

C. Defect Audit of the Released Implementation

The released OptimizedTrafficCLIP repository was executed as documented and inspected line by line. Table II summarizes six defects found in its training and evaluation pipeline. All six were corrected in place, without altering the model architecture itself. Two further unrelated bugs were found and fixed while exercising the corrected pipeline against real data: the number of output classes was derived from the full 20-class taxonomy rather than from the loaded archive's actual label count, and the default experiment-tracking serialization path crashed at the end of every run.

D. Honest Re-Evaluation Protocol

After applying all six corrections, the manuscript's own hypothesis-testing evaluation protocol was re-executed at the manuscript's own reported seed, for every configuration the manuscript reports, allowing a direct comparison against its published table for the equivalent configurations.

E. Independent Bi-Modal Reimplementation (Track A)

A second implementation of the underlying TrafficCLIP architecture was built independently from the published specification, without consulting the released OptimizedTrafficCLIP code, so that no implementation choice of the release could propagate into the reference point.

F. Independent Tri-Modal Reimplementation (Track A-Full)

A further independent implementation extends Track A with OptimizedTrafficCLIP's two distinguishing components, the physics-statistics branch and physics-conditioned dynamic

TABLE II
DEFECTS IDENTIFIED IN THE RELEASED IMPLEMENTATION

#	Defect	Effect
1	Contrastive-loss weight α not exposed as a constructor argument	Manuscript's stated configuration unreachable through the documented API
2	Evaluation forward pass embeds the ground-truth class name in the dynamic prompt	Evaluation-time label leakage that inflates the reported score
3	Benchmark script regenerates the train and test split at a shifted seed on each repeated pass	Split leakage across repeated evaluation passes
4	Hypothesis-testing validation loop processes one sample at a time and reads an unresolved class-name variable instead of the resolved list	Silent incorrectness in the dynamic-prompt evaluation path
5	Training data loader built with shuffling disabled	Frozen batch composition degrades the contrastive loss term
6	Frozen backbone submodules not forced into evaluation mode during training	Dropout remains active and batch-normalization statistics keep drifting despite frozen weights

prompting, fused through a non-linear head. This implementation avoids Defects 1, 2, 3, 4, and 6 of Table II by construction: the data split is fixed once per dataset rather than regenerated per run, frozen submodules are pinned to evaluation mode inside the model's own training routine, and training uses the same batched multi-candidate hypothesis computation used at test time, so the model is never given the sample's true-label prompt as a training input.

Evaluating K candidate prompts through a live text encoder at every training step does not scale; it was measured at approximately 7.5 seconds per step at $K = 20$. Because every sample's physics statistics are fixed for a given run, its K dynamic prompt strings and their text embeddings are fixed as well, so they are precomputed once into an $(N, K, 768)$ cache. An equivalence test asserts that cached and live hypothesis logits agree exactly, which gates every Track A-Full number reported here.

G. Independent Baselines (Track C)

Four additional models were implemented from scratch and trained on the identical data and test split described in Section IV-B: FlowPic [5], FS-Net [6], a 1D convolutional network over the raw byte sequence, and a small Transformer encoder over the same sequence chunked into tokens. All four consume the same 28×28 byte-flow image (equivalently, the 784-byte sequence) used by Tracks A and A-Full. For FlowPic and FS-Net specifically, this substitutes for their own originally published input representations, FlowPic's packet-size/inter-arrival-time histogram and FS-Net's signed per-packet-length sequence. That substitution was closed in a follow-up run: raw packet captures were re-acquired, and both baselines were re-trained on their own published representations, extracted directly from the same underlying flows (Section VI-C). No published FlowPic or FS-Net evaluation exists on USTC-TFC2016 itself, so the comparison is necessarily each baseline's own representation against the byte-image substitution on the same data, not a same-dataset published-versus-reproduced comparison.

H. Cross-Dataset Datasets

To test whether Track A and Track A-Full's parity result is specific to USTC-TFC2016 or generalizes, both tracks and all four Track C baselines were re-run, unmodified beyond the number of output classes, on three additional, independently sourced datasets spanning a different application domain, network generation, and traffic-collection methodology each:

- **ISCXVPN2016** [15]: a 3.1 GB "01" split of the public VPN/non-VPN traffic corpus, re-labeled into a 5-class traffic-type taxonomy (Chat, Email, File Transfer, P2P, VoIP/Video) by folding VPN and non-VPN captures of the same type into one class, so the task remains traffic-type classification rather than VPN-detection. The split is naturally imbalanced (File Transfer: 127 samples, P2P: 487, against 900 each for the three larger classes), reported as-is rather than rebalanced. 2,914 samples total.
- **5G Traffic Capture Dataset** [16]: the 2.3 GB inference split of a University of Juiz de Fora 5G-core dataset, four continuous packet captures labeled by traffic type (eMBB video streaming, URLLC live streaming, mMTC low-rate UDP telemetry). Each capture is a single sustained session on one flow 5-tuple, so a 60-second idle-timeout is applied when segmenting flows (Section IV-B's own protocol uses no timeout, since USTC-TFC2016's captures do not need one); mMTC remains the smallest class at 59 samples, since the corpus itself ships only two mMTC captures, one of them a perfectly constant-rate synthetic generator with no idle gaps to split on. 472 samples total.
- **IoT-23** [17]: a hand-selected, 2.5 GB, 9-class subset (3 real, non-simulated benign IoT devices; 6 malware family captures) of the Stratosphere Laboratory's IoT malware/benign corpus, chosen to fit a small-dataset budget from a 21 GB full release. The same 60-second flow-idle-timeout is applied, which matters for one class specifically: a sustained Mirai DDoS-flood capture yields only 20 usable flow samples without a timeout against 229 with it, since a flood targeting one victim IP is otherwise a single, arbitrarily long same-5-tuple flow. 6,340 samples total, imbalanced by capture availability (Mirai: 229, Somfy Doorlock: 258, Philips Hue: 453, all other classes at the 900-sample cap).

All three datasets use the identical preprocessing, split protocol, and evaluation code as Section IV-B; only the source `.npz` archive and class count change.

I. An Additional Divergence: Visual Stem Initialization

Beyond the six defects of Table II, the release diverges from the pretrained backbone's intended use in a way that the independent tracks handle differently. ResNet-50's first convolutional layer is a $(64, 3, 7, 7)$ kernel trained for three-channel

TABLE III
TRACKS A AND A-FULL ON USTC-TFC2016 (PERCENT ACCURACY, MEAN ± SD, 3 SEEDS)

Configuration	Track A (bi-modal)	Track A-Full (tri-modal)
10-class	99.75 ± 0.09	99.68 ± 0.07
20-class	96.64 ± 0.23	96.63 ± 0.09

input. The release replaces it outright with a freshly initialized single-channel `Conv2d(1, 64, 7, 7)` and then freezes it, so the frozen backbone receives an untrained random first-layer projection. Tracks A and A-Full instead adapt the pretrained kernel to single-channel input by summing its weights across the input-channel dimension, which is equivalent to replicating a grayscale image across the three original channels. Because this was the leading prior hypothesis for the release's underperformance, an ablation variant reinstalls the release's version verbatim inside each otherwise correct codebase, with every other component held fixed, to isolate its individual contribution.

V. EXPERIMENTAL SETUP

All training runs were executed on a single Apple M4 GPU using the Metal Performance Shaders backend. Tracks A and A-Full were each trained for 200 epochs per configuration and averaged over 3 independent seeds. Track C baselines were trained for an equivalent budget and averaged over 5 independent seeds. Track B's honest re-evaluation was likewise re-run across 5 independent seeds (62–66) for all six configurations, matching the manuscript's own protocol at each seed. Pairwise Wilcoxon signed-rank tests with Bonferroni correction were computed across all 15 configuration pairs; at $n = 5$ seeds the exact two-sided test cannot fall below $p = 0.0625$ regardless of effect size, so a null result at this sample size reflects that structural floor rather than a finding of no difference, and is reported as such rather than suppressed. Accuracy and Macro F1 are reported as the primary metrics throughout, since the class distribution is only approximately balanced at the 20-class setting.

VI. RESULTS

A. Tracks A and A-Full: Independent Reimplementations

Table III reports both independent reimplementations under the identical protocol. The tri-modal Track A-Full matches the bi-modal Track A to within 0.1 point at both class counts, so the added physics and dynamic-prompt modalities neither help nor hurt on this dataset once implemented correctly. Accuracy and Macro F1 coincide to within 0.01 point because the test partitions are near-balanced, so only accuracy is shown.

Visual stem ablation. Reinstalling the release's untrained, frozen single-channel stem (Section IV-I) inside each otherwise correct codebase leaves 20-class accuracy unchanged within seed variance: 96.68% ± 0.14 against 96.64% for Track A, and 96.80% ± 0.20 against 96.63% for Track A-Full. This divergence therefore does not explain the release's shortfall.

TABLE IV
TRACK B, MANUSCRIPT-REPORTED AGAINST DEFECT-CORRECTED MEASUREMENT UNDER THE MANUSCRIPT'S OWN PROTOCOL (PERCENT MACRO F1, MEAN ± SD, 5 SEEDS)

Var.	Configuration	Reported	Corrected
V1	Original, no stats	62.21	29.54 ± 1.52
V2	Original, with stats	59.05	31.54 ± 6.43
V3	Optimized, hard prompt	63.10	39.66 ± 4.90
V4	Optimized, dynamic prompt	78.57	**86.19 ± 0.87**
V5	Optimized, hard prompt, stats	67.01	43.83 ± 3.50
Full	Optimized, dynamic prompt, stats	79.63	**88.29 ± 2.15**

B. Track B: Honest Re-Evaluation of the Released Code

Table IV compares the manuscript's own reported values for all six of its configurations under its stated hypothesis-testing protocol against the same configurations re-measured, across 5 seeds each, after the six defects of Table II are corrected. The four configurations that rely on a static hard prompt (V1, V2, V3, V5) collapse once the label-leaking evaluation path is repaired, which is the behavior a genuine leak predicts. The two configurations that use the physics-conditioned dynamic prompt (V4 and Full) instead improve: V4 rises by 7.6 points of Macro F1 (5-seed mean) and Full by 8.7 points. The manuscript's separately reported headline figure of 87.54% originates from a distinct forward-pass evaluation path whose text input carries the ground-truth label, and is not reproduced under the corrected protocol. A pairwise Wilcoxon signed-rank test with Bonferroni correction across all six configurations found no Bonferroni-significant pairs, but as noted in Section V this null result reflects the $n = 5$ floor ($p \geq 0.0625$) rather than genuine equivalence between the dynamic- and static-prompt configurations. A paired t-test with the same correction separates V4 and Full from every static-prompt configuration ($p < 0.001$), while V4 and Full do not differ from each other ($p = 0.93$).

C. Track C: Independent Baselines

Table V reports all four Track C models trained from scratch on the identical data and split. On the shared byte-image representation every model exceeds 99% accuracy at 10 classes and 93% at 20, matching Tracks A and A-Full with four architecturally unrelated families. FlowPic and FS-Net were additionally re-trained on their own originally published input representations, extracted from re-acquired raw packet captures for the identical flows: FlowPic's packet-size/inter-arrival-time histogram and FS-Net's signed per-packet-length sequence. Both score measurably lower on their own representation (FlowPic: 18.7 points lower at 10 classes, 29.2 at 20; FS-Net: 7.8 and 14.5), confirming the direction that Section IV-G's substitution caveat predicted. Neither original paper evaluates on USTC-TFC2016, so no same-dataset published number exists to compare either representation against.

TABLE V
TRACK C BASELINES ON USTC-TFC2016 (PERCENT ACCURACY, MEAN ± SD, 5 SEEDS)

Model	Byte image		Own representation	
	10-class	20-class	10-class	20-class
FlowPic	99.39 ± 0.06	93.87 ± 0.35	80.71 ± 0.06	64.63 ± 0.57
FS-Net	99.45 ± 0.18	95.50 ± 0.29	91.67 ± 0.39	80.96 ± 0.95
1D-CNN	99.63 ± 0.10	96.21 ± 0.22	–	–
Transformer	99.50 ± 0.14	95.17 ± 0.41	–	–

TABLE VI
CROSS-DATASET GENERALIZATION (PERCENT MACRO F1, MEAN ± SD; 3 SEEDS FOR TRACKS A AND A-FULL, 5 FOR TRACK C)

Model	ISCXVPN2016	5G	IoT-23
Track A	92.06 ± 0.96	98.20 ± 1.35	98.23 ± 0.27
Track A-Full	91.47 ± 0.96	98.91 ± 0.89	98.67 ± 0.32
FlowPic	90.88 ± 0.58	97.81 ± 0.00	97.99 ± 0.13
FS-Net	89.54 ± 1.01	98.24 ± 0.90	97.96 ± 0.38
1D-CNN	92.86 ± 0.65	96.91 ± 1.10	98.26 ± 0.18
Transformer	85.00 ± 5.14	95.88 ± 1.58	97.40 ± 0.38

D. Cross-Dataset Generalization

Table VI reports Tracks A and A-Full and all four Track C baselines, unmodified beyond output class count, on the three additional datasets of Section IV-H. Macro F1 is shown because these datasets are imbalanced. On every dataset the two independent tracks remain within one standard deviation of each other, replicating the USTC-TFC2016 parity finding of Section VI-A on a VPN traffic-type task, a 5G network-slice task, and an IoT device/malware-family task. Every model exceeds 85% Macro F1 on every dataset, and the ranking among the baselines is not consistent across datasets (FS-Net leads on 5G, 1D-CNN on ISCXVPN2016 and IoT-23), so no single architecture is uniformly superior across traffic domains. The Transformer's wide spread on ISCXVPN2016 (5.14 points) is the largest in the table and most likely reflects its instability on a small, imbalanced training set, since the same model is within 1.1 points of the 1D-CNN at 20 classes on USTC-TFC2016.

E. Anatomy of a Confusable Pair

Fig. 2 breaks Track A-Full's 20-class result down by class. Nine classes are classified perfectly on every seed, eighteen of twenty are at or above 95%, and the two exceptions are the botnet families Virut (68.40%) and Neris (78.02%). Summed over three seeds, 207 of the model's 273 test errors (75.8%) are confusions between these two classes, from 70.5% to 82.4% on individual seeds.

Why the physics-statistics branch cannot separate them. Both classes contain 900 flows in the dataset. Their per-flow statistics, computed directly from the test split and independent of any model, have nearly identical medians: inter-arrival time 1793 ms against 1783 ms, jitter 2399 ms against 2396 ms, and byte entropy 4.94 against 4.93. The means differ widely (inter-arrival time 181,107 ms against 17,976 ms), but only because of a small number of extreme outliers; the bulk of the two

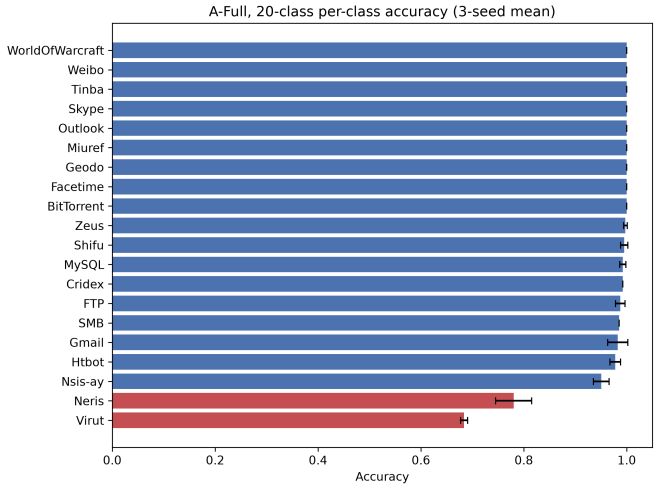


Fig. 2. Track A-Full 20-class per-class accuracy, 3-seed mean with standard deviation. Virut and Neris (highlighted) are the only classes below 95%.

distributions coincides. Since the statistics branch and the dynamic prompt both read exactly these three values, the tri-modal model receives almost no information from them that distinguishes the typical Virut flow from the typical Neris flow, and must rely on the byte image. Once a flow is assigned to one of the two classes, A-Full separates them correctly 74.1% of the time (593 of 800 decisions).

Do additional features help? To test whether other flow properties carry the missing signal, seven features absent from the three-value statistics vector were computed from the raw captures of the same 900 flows per class: packet-size mean, standard deviation, and skewness; burst count and mean burst size at a 1 s idle gap; and byte entropy over only the first 200 and only the last 200 bytes of each flow. A two-sample Welch t-test was applied to each. The most informative features concern where in the flow entropy is measured rather than its overall level: last-200-byte entropy differs strongly between the classes ($t = 9.27$, $p = 5 \times 10^{-20}$), as does first-200-byte entropy ($t = 5.67$, $p < 10^{-7}$), while whole-flow entropy does not. Packet-size mean ($p = 0.003$) and skewness ($p = 0.006$) differ moderately; packet-size spread and burst statistics do not differ significantly ($p \geq 0.08$). A logistic regression on the seven features alone separates the pair with 61.4% ± 4.9% accuracy under 5-fold cross-validation, against a 50% chance level. This is real but modest signal, and on its own it is weaker than the 74.1% that A-Full already achieves from the byte image. Whether the extra features are complementary to the image branch, and would therefore reduce the pair's error when added to the model, requires retraining the tri-modal model with an extended statistics vector for all twenty classes; this was not performed and is left to future work.

Implications for deployment. For eighteen of twenty classes the tri-modal model is at or near ceiling, and an operator can rely on its per-class output directly. For Virut and Neris, roughly one flow in four is attributed to the wrong family. Both are botnets, so a deployment that needs only to flag botnet traffic can merge the two into one alert class with little loss. One that must attribute a flow to the correct

family should treat the pair as a single ambiguous group and resolve it with a second-stage decision or with evidence beyond a single flow, such as host-level behavior across many flows. An aggregate accuracy of 96.6% hides this failure mode entirely, which is why per-class reporting is among the recommendations of Section IX.

VII. Discussion

The results in Section VI support a single central conclusion: OptimizedTrafficCLIP's measured accuracy gap against a defect-free reference is a property of the released implementation, not of the tri-modal architecture it implements. Track A-Full, an independent rebuild of the identical tri-modal design, reaches 99.68% accuracy on the 10-class benchmark, essentially matching Track A's simpler bi-modal design and 11.6 percentage points above Track B's corrected 5-seed mean of 88.13% for the same architecture on the same task.

The visual stem ablation of Section VI-A further shows that the gap cannot be attributed to the single divergence that was most suspected in advance. Replacing the adapted pretrained convolution kernel with an untrained frozen one effectively feeds the frozen backbone a random first-layer projection, yet isolated inside an otherwise correct codebase it costs approximately nothing in either independent track. The most likely explanation is that the trainable detail-aware branch and the trainable adapter and fusion parameters have enough capacity to compensate for a randomized first convolutional layer on a dataset this separable.

Track C's baselines contextualize the finding further. Four architecturally unrelated models, a two-dimensional convolutional network over the byte image, a bidirectional recurrent encoder, a 1D convolutional network, and a small Transformer, all exceed 99% accuracy on the 10-class benchmark when trained from scratch on the same data. The 10-class USTC-TFC2016 task is therefore close to solved by several different architectural families, which is the appropriate context for reading Track B's corrected 88.13% $\pm$ 2.12% accuracy: it reflects this specific implementation's defect profile and training budget on this task, and is not evidence that the task itself is difficult.

Cross-dataset generalization (Section VI-D) extends the central finding beyond a single corpus. Tracks A and A-Full remain statistically indistinguishable on ISCXVPN2016 (a VPN traffic-type task), a 5G traffic dataset (a network-slice task), and IoT-23 (an IoT device/malware-family task), three datasets that differ from USTC-TFC2016 and from each other in application domain, network generation, and collection methodology. The tri-modal design's parity with its simpler bi-modal counterpart is therefore not an artifact of one dataset's specific separability; it replicates on four independent datasets spanning four different classification problems. Track C's four baselines generalize similarly, each exceeding 85% Macro F1 on every dataset with no single baseline uniformly ahead across all three, which argues against reading Track B's gap as a general property of frozen-backbone or CLIP-style approaches on this class of task, since three unrelated architectural families make the same problems look easy regardless of source dataset.

Table IV shows that OptimizedTrafficCLIP's own central contribution, physics-conditioned dynamic prompting, is a real and reproducible mechanism. Under the manuscript's own protocol, correcting the six identified defects raises the Full configuration from a single-run 79.63% to an 88.29% $\pm$ 2.15% 5-seed mean Macro F1 rather than lowering it, and only the two dynamic-prompt configurations clear 80% once the leak is repaired. What does not reproduce is the separately reported 87.54% headline figure, which this audit traces to a distinct evaluation-path defect rather than to the architectural contribution itself.

A. Why the Label Leak Inflated the Reported Score

In OptimizedTrafficCLIP the text branch reads a sentence built from a class name and the flow's physics statistics. At training time the true class name is a legitimate input, because the contrastive objective needs a positive pair. At evaluation time it is not, because the class name is exactly what the model must predict. The released evaluation path that produced the 87.54% figure built each sample's prompt from that sample's own ground-truth class and took the argmax of a single forward pass. The correct class name was therefore present in the input the fusion head read before making its prediction, and the head could recover the label from the text branch without using the visual or statistical evidence at all. The label-free alternative, which the manuscript's own hypothesis-testing protocol specifies, scores all K candidate prompts and selects the best match; this is the protocol under which every corrected number in this paper is measured. The pattern in Table IV is what this mechanism predicts. Configurations whose text input carries little besides the class name (V1, V2, V3, V5) lose between 23 and 33 points once the label is removed, because that name was most of what they relied on. The dynamic-prompt configurations (V4, Full), whose prompts also carry per-sample statistics, do not collapse. In the taxonomy of Kapoor and Narayanan [25] this is leakage from the target into the evaluation-time input. It is easy to miss because the same prompt-construction code is correct in training and wrong in evaluation.

B. Why Independent Verification Matters in This Field

Three results of this study could not have come from re-running the released code alone. First, the corrected release scores 88.13% accuracy at 10 classes, yet a clean-room build of the same architecture reaches 99.7% on the same task. Running only the release would wrongly suggest that the tri-modal design is weak. Second, the byte-image substitution for FlowPic and FS-Net inflates their accuracy by 7.8 to 29.2 points (Table V), so an input representation that is left unstated can move a baseline by more than the gap between competing methods. Third, the residual error of a correct implementation concentrates in one class pair (Section VI-E), which a single aggregate number conceals. Each of these can be caught only by building something independently and comparing. Recent corpus-level evidence points the same way: pretrained encrypted-traffic models lose much of their reported performance once data-preparation shortcuts are removed [29], and

benchmark labels themselves are often derived inconsistently [31]. Independent verification is therefore not a formality in this field; it is how architectural claims are separated from pipeline artifacts.

C. What a Saturated Benchmark Means for the Field

Six models of five kinds (CLIP-style bi-modal and tri-modal models, 2D and 1D convolutional networks, a recurrent encoder, and a Transformer) all exceed 99% on the 10-class USTC-TFC2016 task and lie within 2.8 points of each other at 20 classes (93.9% to 96.6%). On such a benchmark a new architecture cannot show a meaningful improvement in accuracy, and differences of a few tenths of a point sit inside seed-to-seed variance. Claims of novelty on USTC-TFC2016 should therefore rest on something other than headline accuracy: robustness to distribution shift, cost, interpretability, or performance on the hard residual pairs that a saturated aggregate hides. It also follows that a reported shortfall on this benchmark is more likely to indicate an implementation problem than a limitation of the architecture, which is exactly what this study found.

VIII. Limitations

The following limitations bound the conclusions of this study.

- **Statistical power of the Track B significance test.** Track B is averaged over five seeds. At $n = 5$ the exact two-sided Wilcoxon signed-rank test cannot fall below $p = 0.0625$, so the pairwise Wilcoxon test has no power to reject at any Bonferroni-corrected level; the comparison rests on the paired t-test, which assumes approximately normal seed-to-seed differences.
- **Unequal seed counts.** Tracks A and A-Full use three seeds and Tracks B and C use five, set by the per-run compute cost of each track on a single GPU.
- **Track B is evaluated on USTC-TFC2016 only.** The released code is specific to the original 10- and 20-class taxonomy, so the cross-dataset extension covers Tracks A, A-Full, and C but not B.
- **Partly constrained baselines.** FlowPic and FS-Net were re-trained on their own published representations, but the 1D-CNN and the Transformer are generic architectures rather than reimplementations of specific published models, and all baselines use one training budget without per-dataset tuning.
- **Small, imbalanced additional datasets.** The 5G dataset's mMTC class has 59 samples from two source captures and IoT-23's Mirai class has 229, reported as collected rather than rebalanced; the 5G Transformer cell shows a correspondingly wide seed-to-seed spread.
- **Residual-error analysis is limited to one pair.** The additional features of Section VI-E were computed only for Virut and Neris and tested with a standalone classifier; a retrain of the tri-modal model with an extended statistics vector for all twenty classes was not performed, so whether those features would reduce the pair's error inside the model remains open.
- **No adversarial or open-set evaluation.** Robustness to adversarially shaped packet timing or padding, and behavior on traffic classes unseen during training, were not tested.
- **Single hardware platform.** All runs used one Apple M4 GPU with the Metal Performance Shaders backend; run-to-run numerical behavior on CUDA hardware was not checked.

IX. Recommendations for Reproducible Research in Encrypted Traffic Classification

Each defect in Table II and each correction in this study suggests a practice that would have prevented or exposed it. We group them by the stage of the research cycle at which they apply.

A. Code Release

1) **Release one evaluation path, and make it label-free by construction.** The headline figure and the corrected table came from different code paths. A single evaluation entry point whose inputs demonstrably exclude the target (Defect 2) removes this failure mode. A unit test that shuffles labels and checks that accuracy drops to chance is a cheap guard.
2) **Fix the data split once and assert it.** Persist the train, validation, and test indices, or derive them from a single fixed seed, and add a test that fails if two scripts produce different splits (Defect 3). Every comparison in this paper is gated on such a test.
3) **Pin framework modes explicitly.** Frozen submodules should be forced into evaluation mode inside the training loop, and data loaders should shuffle during training (Defects 5 and 6). Neither is visible in the reported numbers, so both should be tested directly.
4) **Expose every hyperparameter the paper reports.** A configuration named in the paper but unreachable through the released interface (Defect 1) cannot be reproduced as described.

B. Reporting

5) **Report multi-seed statistics with the test's power stated.** Report a mean with a standard deviation or confidence interval over at least five seeds. If a significance test is used, state its minimum attainable p-value at the chosen sample size; at least ten paired seeds are needed before a Wilcoxon test can survive a Bonferroni correction over fifteen comparisons.
6) **State input representations exactly.** When a baseline is run on a representation other than its published one, say so and, where possible, report both, since the choice alone moved accuracy by up to 29 points here.
7) **Contextualize against saturated baselines.** Report at least one simple from-scratch baseline on the same split, so that readers can see how much of the benchmark is already solved.
8) **Report per-class results.** An aggregate of 96.6% concealed a pair at 68% and 78%. Per-class accuracy and the

confusion matrix should accompany every multi-class result.

C. Verification

9) **Verify independently before publication.** A clean-room reimplementation from the paper's own description, by someone who has not read the released code, tests both the paper and the code at once. Where it disagrees with the release, the disagreement locates a defect.
10) **Test on at least one additional dataset.** A finding that holds on one benchmark may reflect that benchmark's separability; replication on independently collected data is inexpensive once the pipeline accepts an arbitrary class count.
11) **Publish a reproducibility checklist with the artifact.** A checklist in the style of the NeurIPS program [24], adapted to traffic data (capture source, flow segmentation and timeout, header masking, split procedure, seed list), lets reviewers check the points above without re-running the code.

X. Conclusion

This paper reports an independent reproducibility study of OptimizedTrafficCLIP. A line-by-line audit of the released implementation identified six defects in its training and evaluation pipeline, one of which directly leaks the ground-truth label into the evaluation path. Correcting all six and re-running the manuscript's own evaluation protocol over five seeds raises the Full configuration from a single-run 79.63% to 88.29% Macro F1. The dynamic-prompting mechanism is therefore genuine, while the separately reported 87.54% headline figure is not reproducible. Two independent reimplementations, one bi-modal and one tri-modal, reach 99.7% accuracy on the 10-class benchmark, 11.6 points above the corrected release, which shows that the architecture is not the limiting factor. Four from-scratch baselines exceed 99% on the 10-class task, so the benchmark is largely solved by several unrelated model families. The error that remains in a correct implementation is concentrated in a single pair of botnet families that the physics statistics cannot separate. The bi-modal and tri-modal parity result replicates on three independently sourced datasets covering VPN traffic type, 5G network slices, and IoT device and malware families.

The study's broader output is methodological: a five-stage audit procedure and eleven recommendations that, applied before publication, would have caught every defect reported here. Future work includes re-targeting the released code to the additional datasets so that its gap can be tested beyond USTC-TFC2016, retraining the tri-modal model with an extended statistics vector for all classes, evaluating open-set and adversarial robustness, and applying the same audit procedure to other released encrypted-traffic classifiers.

Code and Data Availability

All code for the corrected release, both independent reimplementations, the four baselines, the dataset extraction scripts, and the per-seed result files underlying every table and figure in this paper are available in a public reproducibility package at `https://github.com/rrsd2769/trafficCLIP`. The four datasets are publicly available from their original sources [3], [15]–[17].

Acknowledgment

The authors thank Pramit De for the original OptimizedTrafficCLIP architecture and manuscript that this study reproduces, validates, and extends, and for guidance throughout this project.

References

[1] R. Chen, L. Luo, D. Guo, X. Wang, S. Li, and C. Qiu, "TrafficCLIP: A lightweight cross-modal framework for network traffic classification," *Comput. Netw.*, vol. 272, Art. no. 111662, 2025.
[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.
[3] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. IEEE ICOIN*, 2017, pp. 712–717.
[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.
[5] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 2, pp. 1218–1232, 2021.
[6] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM*, 2019, pp. 1171–1179.
[7] P. Khosla *et al.*, "Supervised contrastive learning," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 18661–18673.
[8] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.
[9] P. De, "OptimizedTrafficCLIP: A tri-modal cross-modal framework for encrypted network traffic classification with physics-informed prompting and edge deployment," unpublished manuscript, 2026.
[10] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf. (WWW)*, Lyon, France, 2022, pp. 633–642.
[11] N. Wickramasinghe, A. Shaghaghi, G. Tsudik, and S. Jha, "SoK: Decoding the enigma of encrypted network traffic classifiers," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2025.
[12] J. Luxemburk and K. Hynek, "DataZoo: Streamlining traffic classification experiments," in *Proc. ACM CoNEXT Workshop on Explainable and Safety Bounded, Fidelitous Machine Learning for Networking (SAFE)*, 2023.
[13] E. Akbari, Z. Zhou, M. A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "One task to rule them all: A closer look at traffic classification generalizability," arXiv:2507.06430, 2025.
[14] L. Gioacchini, I. Drago, M. Mellia, Z. Ben Houidi, and D. Rossi, "Generic multi-modal representation learning for network traffic analysis," arXiv:2405.02649, 2024.
[15] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2016, pp. 407–414.
[16] L. Azalim de Oliveira, E. Franco Silva, and L. Jerez Chaves, "5G traffic capture dataset: User equipment classification in the 5G core," Zenodo, 2025, doi: 10.5281/zenodo.15064130.
[17] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, CTU Univ., Zenodo, 2020, doi: 10.5281/zenodo.4743746.
[18] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "Yet another traffic classifier: A masked autoencoder based traffic Transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 5420–5427.
[19] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, 2024.

[20] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2025.

[21] E. H. Choi, "FlowCLIP: Contrastive pretraining using domain names for encrypted traffic classification," arXiv:2606.17746, 2026.

[22] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, Art. no. 106944, 2019.

[23] H. Jin *et al.*, "Traffic-CBM: A structurally interpretable multimodal framework for encrypted traffic classification," arXiv:2606.29909, 2026.

[24] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program)," *J. Mach. Learn. Res.*, vol. 22, no. 164, pp. 1–20, 2021.

[25] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023.

[26] X. Bouthillier *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021.

[27] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[28] A. S. Jacobs, R. Beltiukov, W. Willinger, R. A. Ferreira, A. Gupta, and L. Z. Granville, "AI/ML for network security: The emperor has no clothes," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Los Angeles, CA, USA, 2022.

[29] Y. Zhao, G. Dettori, M. Boffa, L. Vassio, and M. Mellia, "The sweet danger of sugar: Debunking representation learning for encrypted traffic classification," in *Proc. ACM SIGCOMM*, 2025.

[30] C. Wang, X. Xie, T. Wang, and Y. Cui, "Bias in the shadows: Explore shortcuts in encrypted network traffic classification," arXiv:2601.10180, 2026.

[31] S. Huang and S. Yang, "SoK: Where do flow labels come from? Auditing label provenance in encrypted traffic benchmarks," arXiv:2609.02140, 2026.

[32] A. Finamore, C. Wang, J. Krolikowski, J. M. Navarro, F. Chen, and D. Rossi, "Replication: Contrastive learning and data augmentation in traffic classification using a Flowpic input representation," in *Proc. ACM Internet Meas. Conf. (IMC)*, 2023.